



Universidad
Rey Juan Carlos

Sistemas Operativos

PRÁCTICA 2 - MINISHELL

LUCÍA DOMÍNGUEZ RODRIGO, MARCOS JIMÉNEZ PULIDO

**TABLA DE CONTENIDO**

Autores	2
Descripción del Código	2
Diseño del Código	2
Variables globales	7
Principales Funciones	7
Casos de Prueba	18
Comentarios Personales	21
Problemas encontrados	21
Críticas constructivas	22
Propuesta de mejoras	22
Evaluación del tiempo dedicado	22
Anexo	23
Diagrama de flujo del programa principal	23
Diagrama de funcionamiento y gestión de procesos en MiniShell	24

Autores

Nombre	Apellidos	DNI	Correo Institucional
Lucía	Domínguez Rodrigo	49452469-P	l.dominguez.2021@alumnos.urjc.es
Marcos	Jiménez Pulido	04860135-M	m.jimenezp.2021@alumnos.urjc.es

Descripción del Código

Diseño del Código

El diseño del código gira en torno a la idea de la gestión continua de procesos en un entorno de terminal, simulando las funcionalidades básicas de una shell de comandos. Se sigue un enfoque modular, el cual permite gestionar procesos en segundo plano, manejar redirecciones y controlar señales.

Arquitectura general del código

El programa se estructura en torno a:

- *Estructura de datos clave*: tales como el tipo struct Tjob, el cual guarda la información de los trabajos que serán gestionados en el código.
- *Bucle principal de la Minishell*: controla el flujo de ejecución de esta (representado mediante un diagrama en [Anexo: Diagrama de flujo del programa principal](#)).
- *Funciones específicas*: módulos especializados en manejo de comandos, señales y redirecciones.

Estructura de datos principal: Tjob

Esta estructura es clave para gestionar los trabajos en ejecución. Sus campos incluyen:

- *ncommands*: Número de comandos de los que se compone el trabajo.
- *pid_array*: array de identificadores de procesos (PIDs).
- *job_id*: identificador único del trabajo.
- *command*: texto que representa el comando completo introducido por el usuario.
- *status*: enumerado que representa los posibles estados de un trabajo: en ejecución (*RUNNING*), parado (*SUSPENDED*), y terminado (*FINISHED*).



- **shown:** indica si el trabajo ya ha sido mostrado al usuario. Actuará como indicador de borrado de ese trabajo de la lista `job_list`, junto al status.

La lista global `job_list` almacena los trabajos en segundo plano, mientras el trabajo `fg_job` gestiona el proceso activo en primer plano.

Cabe destacar que el trabajo en *foreground* se copia a `job_list` cuando es suspendido (es decir, el programa detecta que se ha llamado a la señal `SIGSTOP` con `CRTL+Z`), permitiendo que pueda manejarse como un trabajo en segundo plano si fuese necesario (es decir, si se introduce el mandato `bg` junto al identificador del trabajo).

Gestión de procesos: foreground y background

Los procesos en primer y en segundo plano se gestionan de forma distinta:

- **Primer plano (Foreground):** Los procesos en primer plano se gestionan mediante comprobaciones al struct `fg_job`. El código asegura que solo un trabajo pueda estar en *foreground* en un momento dado.
- **Segundo plano (Background):** Los trabajos enviados al segundo plano se almacenan en `job_list`, con una capacidad máxima de `MAX_JOBS` trabajos simultáneos. Si se sobrepasa esta capacidad, el minishell imprimirá un mensaje por salida estándar y abortará la ejecución de nuevos trabajos. En cuanto a la gestión de memoria de `job_list`, cada vez que se ejecuta un trabajo y se manda a segundo plano se recalcula la siguiente posición de `job_list` a llenar (`next_job`), posibilitando la opción de ejecutar trabajos en casillas de `job_list` no consecutivas.

Cuando un comando es suspendido, se mueve el trabajo en `fg_job` a `job_list`, con el estado actualizado a `SUSPENDED`. Cuando un comando finaliza, mediante el control de la señal `SIGCHLD` se actualiza el estado en `job_list` y marca el trabajo como terminado y a la espera de ser mostrado (mediante el mandato `jobs`).

Manejo de señales

El código responde a señales del sistema para controlar procesos:

- **SIGINT (Ctrl+C):** Finaliza el proceso en *foreground* mediante `fg_handler`.
- **SIGTSTP (Ctrl+Z):** Suspende el proceso en *foreground* y lo mueve a `job_list` con estado `SUSPENDED`.
- **SIGCHLD:** Detecta procesos hijos terminados y actualiza sus estados en `job_list`.
- **SIGCONT:** Detecta procesos en suspensión que quieren ejecutarse en segundo plano.



- **SIGKILL:** Finaliza forzosamente la ejecución de los procesos hijos en el caso de ejecutar el mandato exit.

Redirecciones de entrada salida

El código permite redirigir la entrada, salida y errores:

- **Entrada:** La función *input_redirect* redirige la entrada desde un archivo.
- **Salida:** *output_redirect* redirige la salida a un archivo, sobrescribiendo su contenido.
- **Errores:** *error_redirect* redirige la salida de error a un archivo especificado

Soporte y gestión de pipes

Para manejar comandos encadenados con '|', se crea un array dinámico de pipes (*pipe_array*). Cada subcomando se conecta mediante pipes para intercambiar datos de entrada y salida. Cada elemento del array representa un par de descriptores de archivos (lectura y escritura) que concatenan la salida estándar de un subcomando con la entrada estándar del siguiente.

En cada proceso hijo generado con *fork()* se duplica el descriptor de archivo adecuado utilizando *dup2* para redirigir la entrada/salida según el comando que esté ejecutándose:

- El primer comando redirige su salida al extremo de escritura del primer pipe.
- Los comandos intermedios redirigen su entrada al extremo de lectura del pipe anterior y su salida al extremo de escritura del siguiente pipe.
- El último comando redirige su entrada al extremo de lectura del último pipe.

Para evitar fugas de recursos, cada proceso cierra todos los extremos de los pipes que no utiliza:

- Un comando que escribe en un pipe cierra el extremo de lectura de ese pipe.
- Un comando que lee de un pipe cierra el extremo de escritura de ese pipe.

Una vez que todos los procesos han sido creados y ejecutados, y siempre que la última línea de mandatos haya necesitado el uso de pipes, el programa libera la memoria asignada a *pipe_array* para evitar fugas en cada una de las ejecuciones de mandatos.

Gestión de grupos de procesos

El manejo de grupos de procesos es una funcionalidad fundamental en la MiniShell para coordinar la ejecución y control de múltiples procesos relacionados, como aquellos que forman parte de un pipeline (secuencia de mandatos encadenados por pipes) o un mandato en ejecución. Este diseño permite que la shell controle varios procesos simultáneamente como un solo proceso, facilitando la administración de trabajos en primer plano (*foreground*) y en segundo plano (*background*).



Un **grupo de procesos** es un conjunto de procesos que comparten un identificador común, llamado **PGID** (*Process Group ID*). Este grupo tiene las siguientes propiedades:

- Todos los procesos del grupo reciben las mismas señales colectivamente, como *SIGINT* (*Ctrl+C*) o *SIGTSTP* (*Ctrl+Z*).
- El primer proceso en un grupo suele ser el líder, y su PID se utiliza como PGID.
- Se evita la reacción indebida de otros procesos ante la transmisión de señales, de forma que solamente la reciban los procesos requeridos.

En cuanto a los puntos clave de dichos grupos en el código, se tienen:

1. Creación del Grupo de Procesos

Cuando se ejecuta un comando o un pipeline, se asigna un **PGID único** al grupo:

- El primer proceso en ejecutarse (líder del pipeline) crea el grupo con *setpgid*, utilizando su propio PID como PGID.
- Los procesos adicionales del pipeline se unen al grupo existente.

2. Control del Terminal

Solo un grupo de procesos puede tener el control del terminal en un momento dado.

3. Señales a Grupos de Procesos

La MiniShell utiliza señales para coordinar y controlar los grupos de procesos:

- **Suspender:** Con *kill(-pgid, SIGTSTP)*, se suspende todo el grupo en *foreground*, moviéndolo a segundo plano con estado *SUSPENDED*.
- **Terminar:** Con *kill(-pgid, SIGKILL)*, se finalizan todos los procesos de un pipeline o trabajo.

4. Integración con Foreground y Background

Foreground:

- El trabajo en *foreground* (almacenado en *fg_job*) es el único que tiene control del terminal y puede recibir señales directamente del usuario.
- Si se suspende con *Ctrl+Z*, se mueve de *fg_job* a *job_list* con estado *SUSPENDED*.

Background:

- Los grupos de procesos de un trabajo en segundo plano no tienen control del terminal, pero pueden ejecutarse simultáneamente.

- La reanudación de un trabajo suspendido en segundo plano (*bg*) reactiva todo el grupo de procesos asociado.

Flujo del Programa

- **Prompt:** El bucle principal muestra un prompt que incluye el directorio actual.
- **Lectura y Tokenización:** Lee la entrada del usuario. En caso de que la entrada sea una línea vacía no se tokeniza el input. En caso de que contenga comandos, se procesa con `tokenize`.
- **Procesamiento:** Determina si el comando es interno o externo, y realiza las comprobaciones necesarias: que en caso de haber más de un comando en la línea ninguno de ellos sea interno, y que en el caso de la ejecución de los mandatos `jobs` y `bg` la lista `job_list` no esté vacía.
- **Ejecución:** Procesa los comandos y actualiza `job_list` o `fg_job` según corresponda.
- **Actualización y liberación:** Se comprueba el estado de los trabajos tanto en primer como en segundo plano, se actualiza su información dependiendo del resultado de la comprobación y se restablecen y liberan los recursos de memoria estática y dinámica que se han usado para la ejecución anterior, estando disponibles nuevamente para la siguiente línea de mandatos que se ejecute.

Manejo de Errores y Excepciones

El código maneja situaciones como:

- Archivos inexistentes en redirecciones.
- Identificadores de trabajo no válidos o inexistentes en mandatos como *bg*.
- Memoria insuficiente al asignar recursos dinámicos en `job_list`.
- Control de líneas vacías pasadas por `cmd`.

Gestión de Memoria

La memoria dinámica se gestiona cuidadosamente:

- `job_list` y `fg_job` se inicializan al inicio y se liberan al finalizar el programa. Cada trabajo se llena solamente si es necesario y devuelven sus datos a los valores por defecto para evitar sobreescrituras incorrectas.
- Cada trabajo en `job_list` libera sus recursos al terminar la ejecución de la MiniShell.
- El array de pipes (`pipe_array`) se libera tras cada ejecución

Variables globales

Nombre	Tipo	Descripción
job_list	Puntero a Tjob	Estructura para almacenar la información de los trabajos en segundo plano dentro de la minishell
pipe_array	Puntero a punteros de enteros	Array de pipes necesarios para la comunicación entre mandatos
next_job	int	Siguiente posición libre para escritura de job_list
last_job	int	Posición del último job almacenado
prev_last_job	int	Posición del penúltimo job almacenado
parsed_line	Puntero a tline	Posición del penúltimo job almacenado
fg_job	Tjob	Información del único proceso en ejecución en foreground

Principales Funciones

	main	Nombre	Tipo	Descripción
Argumentos	-			
Variables Locales	Variable 1	input	Array estático de char	Buffer para almacenar la entrada del usuario.
	Variable 2	i	int	Contador para diferentes bucles.
	Variable 3	valid	bool	Indicador de que el comando tokenizado se le pueda pasar a process command.
	Variable 4	current_dir	Array estático de char	Buffer para almacenar el directorio actual.
	...			



Valor Devuelto			int	0 en caso de que la ejecución haya sido correcta
Descripción de la Función				Bucle general del programa donde se inicializan las variables principales y se procesan los comandos.

	process_command	Nombre	Tipo	Descripción
Argumentos	Argumento 1	cmd	Puntero a tline	La línea tokenizada
Variables Locales	Variable 1	selected_job	int	Job id del trabajo introducido por el usuario
	Variable 2	i	int	Variable para iteración de bucles y acceso a variables con índices (como pipe_array).
	Variable 3	j	int	Variable de iteración para creación de procesos hijos
	Variable 4	p	int	Variable que muestra los PIDs de cada proceso.
	Variable 5	status	int	Estado del proceso hijo tras un cambio de estado
	Variable 6	job_status	Puntero a char	Guarda el string para imprimir por pantalla el estado del proceso según su valor del enumerado.
	Variable 7	pgid	pid_t	Identificador del grupo de proceso. Permite tratarlos como unidades lógicas
	Variable 8	pid	pid_t	Identificador del proceso. Variable para crear, rastrear y gestionar procesos individuales
Valor Devuelto			void	No devuelve nada ya que procesa el comando.
Descripción de la Función				Maneja comandos internos y externos. Para comandos externos, maneja creación de procesos hijos y su sincronización gestionando los pipelines.



				Soporta la ejecución en segundo plano, visualización de trabajos activos y contiene una opción de depuración.
--	--	--	--	---

	input_redirect	Nombre	Tipo	Descripción
Argumentos	Argumento 1	file	Puntero constante de tipo char	Nombre o ruta del archivo desde el cual se va a leer la entrada
Variables Locales	Variable 1	fd	int	Descriptor de archivos que se obtiene al abrir el archivo especificado
Valor Devuelto			void	-
Descripción de la Función				Redirige la entrada estándar de un proceso para que lea de un archivo en lugar de la consola. Si el archivo no puede abrirse, la función muestra un mensaje de error y termina el proceso.

	output_redirect	Nombre	Tipo	Descripción
Argumentos	Argumento 1	file	Puntero constante de tipo char	Nombre o ruta del archivo en el cual se va a redirigir la salida estándar. Este archivo se creará o se abrirá si ya existe.
Variables Locales	Variable 1	fd	int	Descriptor de archivos que se obtiene al abrir el archivo especificado
Valor Devuelto			void	-
Descripción de la Función				Redirige la salida estándar de un proceso para que escriba en un archivo en lugar de mostrarlo en la terminal. Si el archivo no puede abrirse o crearse, muestra un mensaje de error y termina un proceso.

	error_redirect	Nombre	Tipo	Descripción
--	----------------	--------	------	-------------

Argumentos	Argumento 1	file	Puntero constante a char	Nombre o ruta del archivo en el cual se va a redirigir tanto la salida estándar como la salida de error estándar del proceso
Variables Locales	Variable 1	fd	int	Descriptor de archivos que se obtiene al abrir el archivo especificado
Valor Devuelto			void	-
Descripción de la Función				Redirige tanto la salida estándar como la salida de error estándar de un proceso a un archivo. Si el archivo no puede abrirse, muestra un mensaje de error y termina el proceso.

	run_cd	Nombre	Tipo	Descripción
Argumentos	Argumento 1	cmd	Puntero a tline	Línea tokenizada
Variables Locales	Variable 1	prev_dir	Array estático de char	Almacena el directorio anterior. Se usa para implementar la funcionalidad de cd -, que cambia al directorio anterior.
	Variable 2	current_dir	Array estático de char	Almacena el directorio actual en el momento de la llamada a la función. Se usa para guardar el directorio antes de cambiar a uno nuevo, y para mostrar en el directorio después de haber cambiado.
	Variable 3	directory	Puntero a char	Contiene la ruta del directorio al que quieres ir. Dependiendo de los argumentos proporcionados al comando cd, puede ser: • HOME si no se especifican argumentos. • El directorio anterior si se usa el argumento -. • La ruta expandida formada si se usa ~.



				El directorio directamente especificado por el argumento del comando.
	Variable 4	home	Puntero a char	Puntero a la cadena que contiene la ruta del directorio HOME del usuario. Se usa cuando el argumento del comando cd comienza con ~, para expandirlo a la ruta absoluta.
	Variable 5	expanded_path	Array estático de char	Almacena el directorio resultante de ejecutar el comando con la expansión ~ al valor de la variable HOME más el resto de la ruta proporcionada por el usuario
Valor Devuelto			void	-
Descripción de la Función				Implementación del comando cd para cambiar el directorio de la Shell. Maneja varios casos, como por ejemplo cambiar al directorio HOME. Si el cambio es exitoso, muestra el nuevo directorio y guarda el anterior para futuras referencias. Si no, muestra el correspondiente mensaje de error.

	run_umask	Nombre	Tipo	Descripción
Argumentos	Argumento 1	cmd	Puntero a tline	Línea tokenizada
Variables Locales	Variable 1	current_umask	mode_t	Almacena el valor actual de la máscara
	Variable 2	fincadena	Puntero a char	Se utiliza en la función strtol para marcar el primer carácter no válido encontrado durante la conversión de la cadena a un número. Si la conversión es exitosa, fincadena debe apuntar a '\0', indicando que la cadena fue completamente convertida.
	Variable 3	umask_value_str	Puntero constante a char	Contiene el argumento proporcionado para cambiar la máscara de usuario. Es el

				valor que el usuario quiere establecer como nuevo umask.
	Variable 4	umask_value	long	Almacena el valor numérico (en octal) de la nueva máscara de usuario, que se obtiene convirtiendo la cadena proporcionada en el argumento
	Variable 5	new_umask	mode_t	Almacena el valor de la nueva máscara de usuario después de convertir y validar el argumento.
Valor Devuelto			void	-
Descripción de la Función				Implementa el comando umask, que permite al usuario visualizar o cambiar la máscara de usuario actual. Si no se proporciona un argumento, la función muestra el valor actual de la máscara. Si se proporciona un argumento, valida que sea un número octal válido y, si lo es, cambia la máscara de usuario al valor proporcionado. Si el argumento no es válido, muestra un mensaje de error.

	run_exit	Nombre	Tipo	Descripción
Argumentos	-			
Variables Locales	Variable 1	i	int	Variable utilizada para iteración dentro de bucles
	Variable 2	j	int	Variable utilizada para iteración dentro de bucles
	Variable 3	status	int	Almacena el estado del proceso hijo cuando se espera con la función waitpid. Recibe el valor del estado de salida del proceso hijo después de que termina su ejecución.
Valor Devuelto			void	-
Descripción de la Función				Recorre el job_list, mata los procesos hijos asociados a cada trabajo, espera a que finalicen y libera la memoria utilizada por las estructuras

				correspondientes. Esto asegura una salida limpia del programa, liberando recursos y terminando los trabajos pendientes.
--	--	--	--	---

	<code>fg_handler</code>	Nombre	Tipo	Descripción
Argumentos	-			
Variables Locales	Variable 1	i	int	Índice para recorrer los procesos en <code>fg_job.pid_array</code>
Valor Devuelto			void	-
Descripción de la Función				Finaliza todos los procesos de un trabajo en primer plano que está en ejecución. Si el trabajo está activo (es decir, <code>fg_job.status</code> es <code>RUNNING</code> y tiene procesos asociados), se envía una señal <code>SIGKILL</code> a todos los procesos en el trabajo para terminarlos inmediatamente. Esto asegura que el trabajo en primer plano pueda ser terminado correctamente cuando sea necesario. Es el encargado de controlar la señal <code>CRTL+C</code> .

	<code>stop_handler</code>	Nombre	Tipo	Descripción
Argumentos	-			
Variables Locales	-			
Valor Devuelto			void	-
Descripción de la Función				Si el trabajo en primer plano está en ejecución, la función envía la señal <code>SIGTSTP</code> al grupo de procesos de ese trabajo (usando el PID del primer proceso de <code>fg_job</code> y negándolo para enviar la señal a todo el grupo). Esto detiene todos los procesos del trabajo.

	check_jobs	Nombre	Tipo	Descripción
Argumentos	-			
Variables Locales	Variable 1	i	int	Índice de iteración para recorrer trabajos
í	Variable 2	j	int	Índice de iteración para recorrer los procesos de cada trabajo
	Variable 3	k	int	Índice de iteración para comparaciones y verificaciones
	Variable 4	exit_status	int	Estado de salida de los procesos hijos. Determina si el proceso termina correctamente o por una señal.
	Variable 5	job_id_counter	int	Contador de identificadores de trabajo. Comienza en 1 y se incrementa cada vez que se asigna un nuevo job_id a un trabajo. Asigna un id único a cada trabajo en job_list.
	Variable 6	pid	pid_t	Almacena el PID de un proceso hijo. Se utiliza para verificar el estado del proceso.
	Variable 7	all_finished	bool	Se utiliza para comprobar si todos los trabajos han terminado correctamente. Se inicializa a true, y en caso de que algún proceso
Valor Devuelto			void	-
Descripción de la Función				La función recorre todos los trabajos en job_list y verifica si han terminado. Si lo han hecho: • Si el proceso terminó correctamente (WIFEXITED), el estado del trabajo cambia a FINISHED. • Si terminó por una señal (WIFSIGNALED), el trabajo se elimina.

				Además, organiza el job_id para reflejar el orden de llegada de los trabajos en la lista.
--	--	--	--	---

	<u>next_overwritable_job</u>	Nombre	Tipo	Descripción
Argumentos	-			
Variables Locales	Variable 1	i	int	Índice de iteración utilizado para recorrer los trabajos en el arreglo job_list
Valor Devuelto			void	-
Descripción de la Función				Tiene como objetivo encontrar el primer trabajo terminado y vacío en la lista de trabajos, para poder reutilizar ese espacio para un nuevo trabajo. Si no se encuentra tal espacio, se informa de un error.

	<u>fill_job</u>	Nombre	Tipo	Descripción
Argumentos	Argumento 1	job	Puntero a Tjob	Apunta a un trabajo de tipo Tjob, el cual será llenado con la información obtenida de parsed_line
	Argumento 2	parsed_line	Puntero a tline	Apunta a la línea ya tokenizada con toda la información necesaria para el comando que queremos meter en la lista
Variables Locales	Variable 1	j	int	Variable de iteración
	Variable 2	k	int	Variable de iteración
Valor Devuelto			void	-
Descripción de la Función				Prepara un trabajo con el comando completo (y sus posibles parámetros) y lo marca como en ejecución (RUNNING). También reserva espacio para los procesos que estarán asociados con los comandos del trabajo dependiendo de la



				información recogida en la línea tokenizada.
--	--	--	--	--

	free_job	Nombre	Tipo	Descripción
Argumentos	Argumento 1	job	Puntero a Tjob	Puntero que apunta al
Variables Locales	-			
Valor Devuelto			void	-
Descripción de la Función				Limpia y libera los recursos asociados a un trabajo, garantizando que se liberen adecuadamente las memorias utilizadas para los identificadores de procesos y que se reseteen las propiedades internas del trabajo.

	sigchld_handler	Nombre	Tipo	Descripción
Argumentos	-			
Variables Locales	Variable 1	saved_errno	int	Guarda el valor actual de errno antes de realizar cualquier operación que pueda modificarlo. Es importante guardarlo porque las funciones de sistema como waitpid pueden modificar errno, por lo que es necesario restaurarlo al final de la función.
	Variable 2	status	int	Almacena el estado de salida del proceso hijo que ha terminado
	Variable 3	i	int	Índice de iteración
	Variable 4	pid	pid_t	almacena el identificador del proceso hijo que ha terminado, que es retornado por la función waitpid.
Valor Devuelto			void	-

Descripción de la Función				Maneja la señal SIGCHLD generada cuando un proceso hijo termina. Recolecta los procesos terminados de forma no bloqueante, actualiza el estado de los trabajos en la lista de trabajos (job_list) para reflejar que el proceso ha terminado, y luego restaura el valor de errno al final para evitar efectos secundarios en otras partes del programa.
---------------------------	--	--	--	--

	is_job_list_empty	Nombre	Tipo	Descripción
Argumentos	-			
Variables Locales	Variable 1	i	int	Índice de iteración
Valor Devuelto			bool	Devuelve true si todos los trabajos están vacíos, es decir la lista está vacía. False en caso contrario.
Descripción de la Función				Recorre la lista de trabajos (job_list) y comprueba si algún trabajo está activo o en ejecución (es decir, si tiene comandos, no está finalizado o tiene procesos asociados). Si encuentra alguno de estos trabajos, devuelve false (la lista no está vacía). Si todos los trabajos están vacíos, es decir, finalizados sin procesos asociados, devuelve true (la lista está vacía).

	is_empty_or_space	Nombre	Tipo	Descripción
Argumentos	Argumento 1	str	Puntero a char	Puntero a una cadena de caracteres terminada en nulo que se evaluará para verificar si está vacía o solo contiene espacios.
Variables Locales	Variable 1	str	Puntero a char	Apunta al carácter actual de la cadena que se está verificando durante la iteración.



Valor Devuelto			bool	Devuelve true si la cadena está vacía o solo contiene espacios. Devuelve false si contiene algún carácter no blanco.
Descripción de la Función				Recorre la cadena str carácter por carácter y comprueba si todos son espacios en blanco (según isspace). Si encuentra algún carácter que no sea un espacio, retorna false. Si no hay caracteres o todos son espacios, retorna true.

Casos de Prueba

Para garantizar el correcto funcionamiento del minishell, se han llevado a cabo casos de prueba incrementales para cada tipo de mandato por separado (comandos internos y externos). Concretamente:

- **Mandatos internos:** Puesto que no se contempla su correcto funcionamiento con pipes, se limita y prueba con un solo mandato interno con redirecciones de entrada y de salida. En cuanto a las redirecciones de entrada, ninguno de los mandatos internos implementados recoge información de la salida estándar, por lo que el resultado de redirigir la entrada a un fichero es el mismo que el que se obtiene si se omite esta redirección.

Línea de mandatos ejecutada	Fichero de entrada	Fichero de salida	Salida
cd < input.txt	input.txt	-	Directorio actual: /home/marcos
cd ./parser > output.txt	-	output.txt (stdout)	Directorio actual: /home/marcos/Minishell/minishell/parser
umask < input.txt > output.txt	input.txt	output.txt (stdout)	Valor de máscara (umask) actual: 0002
umask 0123	-	-	Máscara (umask) cambiada a: 0123
jobs	-	-	-
bg 100 < input.txt	input.txt	-	bg: el trabajo 100 no existe
bg > output.txt	-	output.txt (stdout)	bg: el trabajo actual no existe
exit	-	-	Saliendo de la MiniShell...
cd wc -l > output.txt	-	output.txt (stdout)	No pueden ejecutarse mandatos internos con pipes

- **Mandatos externos:** Al contrario que los anteriores, este tipo de mandatos sí pueden ir encadenados por pipes, por lo que sucede que:
 - La entrada estándar se conecta con el extremo de lectura del primer mandato.

- Los mandatos intermedios conectan su entrada con el extremo de lectura del pipe que conecta con el mandato anterior, y su salida con el extremo de escritura del pipe siguiente.
- El último mandato conecta su extremo de escritura con la salida estándar.

En los casos de prueba de este tipo, se intercalan mandatos internos para comprobar su funcionamiento cuando `job_list` no está vacía (es decir, cuando `is_job_list_empty()` devuelve false).

Línea de mandatos ejecutada	Fichero de entrada	Fichero de salida	Salida
<code>ls -la</code>	-	-	total 104 drwxrwxr-x 4 user user 4096 dic 18 00:51 . drwxrwxr-x 6 user user 4096 dic 5 11:12 .. -rw-rw-r-- 1 user user 123 dic 5 11:12 CMakeLists.txt -rwxrwxr-x 1 user user 94 dic 13 09:59 compila.sh drwxrwxr-x 2 user user 4096 dic 18 00:51 .idea -rw-rw-r-- 1 user user 748 dic 16 09:57 input.txt -rwxrwxr-x 1 user user 37192 dic 18 00:51 minishell -rw-rw-r-- 1 user user 33300 dic 18 00:51 minishell.c drwxrwxr-x 2 user user 4096 dic 6 17:13 parser
<code>ls -la directorio_inexistente >& error.txt</code>	-	error.txt (stderr)	ls: no se puede acceder a 'directorio_inexistente': No existe el archivo o el directorio
<code>sort < input.txt > output.txt</code>	<u>input.txt</u>	output.txt (stdout)	-1. A veces, las respuestas más simples surgen de las preguntas más complejas. -2. La lógica del cálculo lambda transforma la abstracción en claridad pura. -3. El sol se oculta lentamente, pintando el cielo con tonos de fuego y miel. -4. Compartir ideas con otros es como encender una vela con otra, sin perder la llama original. -5. Escuchar el silencio es una forma de encontrar las respuestas que el ruido oculta. -6. Aprender algo nuevo cada día es como plantar semillas en un jardín infinito. -7. Un café caliente en una mañana fría puede ser la chispa perfecta para empezar el día. -8. La curiosidad es el motor que impulsa el descubrimiento y la creatividad. -9. Los caminos más difíciles suelen llevar a los destinos más gratificantes.
<code>wc -l < input.txt</code>	<u>input.txt</u>	-	8
<code>ls -la wc -l > output.txt</code>	-	output.txt (stdout)	11
<code>sort input.txt head -n3 wc > output.txt</code>	-	output.txt (stdout)	3 40 236
<code>sort input.txt head -n3 wc > output.txt &</code>	-	output.txt (stdout)	[3] 7584
<code>find / -name hola grep h</code>	-		find: '/snap/core20/1081/etc/ssl/private': Permiso denegado

			find: '/snap/core20/1081/root': Permiso denegado find: '/snap/core20/1081/var/cache/ldconfig': Permiso denegado find: '/snap/core20/1081/var/cache/private': Permiso denegado find: '/snap/core20/1081/var/lib/private': Permiso denegado ...
cat < input.txt wc -c sort -n head -n 5 tee output.txt >& error.txt	-	output.txt (stdout)	748
cat input.txt cut -d- -f2 rev sort head -n 5 > output.txt	-	output.txt (stdout)	.aíd le razepme arap atcefrep apsihc al res edeup aírf anañam anu ne etneilac éfac nU .7 .arup dadiralc ne nóiccartsbá al amrofsnart adbmal oluclác led acigól aL .2 .atluco odiur le euq satseupser sal rartnocne ed amrof anu se oicnelis le rahcucsE .5 .dadvitaerc al y otheimirbucsed le aslupmi euq rotom le se dadisoiruc aL .8 .lanigiro amall al redrep nis ,arto noc alev anu rednecne omoc se sorto noc saedi ritrapmoC .4

Comentarios Personales

Problemas encontrados

Tras realizar las funciones básicas de la práctica, procedimos a implementar las funcionalidades más complejas, de donde surgieron los siguientes problemas:

Problema de compilación con librería estática y ejecutables PIE

El problema surge debido a la incompatibilidad entre **librerías estáticas no-PIC** y ejecutables **PIE**, que son el estándar por defecto en distribuciones modernas. La solución ideal es recompilar la librería con `-fPIC`. Tras intentar ejecutar la compilación del proyecto, nos daba un error relativo a este problema. Después de una extensa búsqueda de información, llegamos a la conclusión de que la mejor opción era recompilar el código del parser. Sin embargo, como no teníamos acceso al código fuente, se optó inicialmente por deshabilitar PIE usando la opción `-no-pie` al compilar el ejecutable.

Finalmente, para solucionar este problema decidimos añadir la opción `-static` al final de la línea de compilación, quedando definitivamente establecida como: `gcc -Wall -Wextra -o myshell myshell.c -L./parser -lparser -l./parser -static`. Cabe destacar la presencia en esta línea de rutas relativas, debido a la organización de los ficheros de la librería que contiene el parser en una carpeta específica llamada "parser".

Job list

Al crear la lista e introducir en ella un mayor número de elementos, nos dimos cuenta de que no la inicializábamos de manera correcta. Por ello, nos daba constantemente *segmentation fault*, debido a la mala inicialización del espacio de memoria reservado para la misma. Además, fue muy difícil visualizar en cada momento el contenido de esta, por lo que tuvimos que implementar una función adicional "debug" para comprobar en todo momento qué procesos se estaban guardando en ella y su información (PIDs de los hijos, estado, comando correspondiente, estado mostrado o pendiente de mostrar, etc.). **Dicha funcionalidad fue preguntada con anterioridad a los profesores** de la asignatura (al no venir indicada en el enunciado), **dando su visto bueno para dejarla incluida en el proyecto final**.

Tratamiento de procesos en suspensión

Al implementar el control de señales de una manera simple (es decir, mandando la señal correspondiente a los procesos hijos) nos dimos cuenta de que el programa no era capaz de diferenciar entre el proceso de ejecución de la propia Shell con el proceso en *foreground*, cuya consecuencia fue que al hacer `ctrl+z` finalizaba el proceso padre de la minishell. Tras investigar los distintos tipos de señales que podíamos utilizar en el `waitpid`, nos dimos cuenta de que podíamos controlar este problema si gestionábamos cada proceso como un grupo, y además controlábamos el estado con la señal `waitpid` con la macro `WUNTRACED`. De esta manera, fuimos capaces de poder establecer las señales de manera correcta, de tal manera que suspendiese el proceso en *foreground* (mandando a la correspondiente señal a cada uno de los hijos) sin matar a la shell.

Tratamiento de señales a procesos hijos

Para establecer el comportamiento de los procesos hijos ante la llegada de señales, nos encontramos con que, al establecer una señal como ignorada, no era posible volver a cambiar ese comportamiento. Tuvimos entonces que recurrir al uso de manejadores para cada señal, además de establecer el comportamiento por defecto de esas señales en los procesos hijos.



Tratamiento de líneas que sólo contienen espacios o enter sin input en prompt

Tras realizar varias pruebas, nos dimos cuenta de que, si al tokenizar una línea sólo se introducía como input espacios, o en su defecto se le daba a Enter directamente, el programa accedía a posiciones de memoria inadecuadas, dando *segmentation fault*. Por ello, antes de tokenizar la línea decidimos verificar si la cadena que se le pasa al input está vacía o formada únicamente de espacios en blanco (tabulaciones, espacios, saltos de línea, etc.). Dicha funcionalidad la aporta la función *is_empty_or_space*, que utiliza la librería estándar *<ctype.h>*, que proporciona funciones para realizar operaciones comunes relacionadas con la clasificación y manipulación de caracteres.

Críticas constructivas

A pesar de que el enunciado especifica las funcionalidades, en ciertas situaciones no termina de quedar clara la interpretación requerida para los comandos. Por ejemplo: al ejecutar un proceso con pipes, no indica (a excepción de *umask*) que, si se detecta un comando interno, no debe de ejecutarse ese proceso.

Propuesta de mejoras

Creemos que, por lo indicado en el anterior apartado, sería conveniente especificar un poco más el enunciado, quedando claras las funcionalidades requeridas (incluyendo excepciones de dichas funcionalidades).

Evaluación del tiempo dedicado

En total, en codificación hemos dedicado unas 70 horas aproximadamente, de las cuales la mayoría han sido corrigiendo los problemas anteriormente mencionados. En documentación y reestructuración del código para que siga buenas prácticas de programación hemos dedicado 10 horas aproximadamente.

Anexo

Diagrama de flujo del programa principal

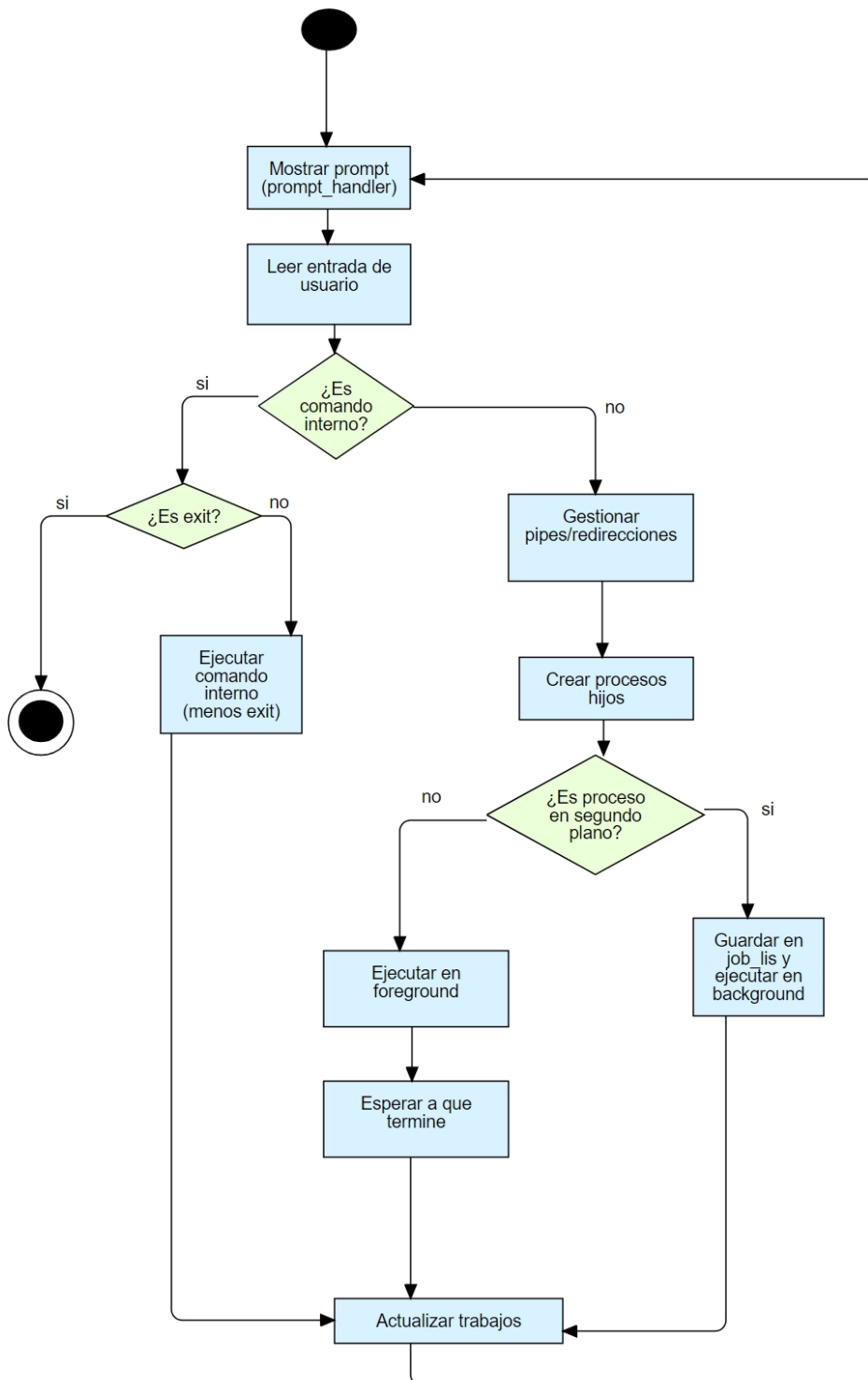


Diagrama de funcionamiento y gestión de procesos en MiniShell

